МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

ЛЕКЦІЯ 2. РЕГРЕСІЯ ТА ГРАДІЄНТНИЙ СПУСК

Львів -- 2026

Лекція зі штучного інтелекту 2026-02

Вступ

На попередній лекції ми оглянули парадигми штучного інтелекту та познайомилися з таксономією машинного навчання. Тепер настав час заглибитися у найпоширенішу парадигму — **навчання з учителем** (*supervised learning*) — і зрозуміти, як воно працює «під капотом».

Навчання з учителем — це задача побудови моделі, яка вчиться передбачувати відповідь на основі прикладів. Уявіть, що ви — геймдизайнер, і хочете автоматично оцінювати складність рівня за його параметрами: кількість ворогів, розмір арени, час на проходження. Ви збираєте дані від тестерів — для кожного рівня знаєте параметри і реальну оцінку складності. Навчання з учителем дозволяє побудувати модель, яка навчиться цей зв'язок відтворювати.

У цій лекції ми зосередимося на **регресії** — задачі передбачення неперервних величин — та на **градієнтному спуску** — універсальному методі оптимізації, що лежить в основі тренування всіх моделей машинного навчання.

Теми, що розглядаються

1. Математична постановка задачі навчання з учителем
2. Лінійна та поліноміальна регресія
3. Функція втрат MSE
4. Градієнтний спуск: batch, stochastic, mini-batch

Математична постановка задачі

Задача навчання з учителем формулюється так: у нас є набір даних (*dataset*) із N прикладів:

$$D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$$

де x_i — вектор ознак (*features*), а y_i — цільова змінна (*target*), тобто «правильна відповідь».

Кожен приклад x_i — це точка у багатовимірному **просторі ознак** (*feature space*). Якщо ми описуємо гравця двома ознаками (час гри, кількість покупок), то кожен гравець — це точка на площині. Якщо ознак три — точка у тривимірному просторі. Для n ознак — точка у n-вимірному просторі, який ми вже не можемо візуалізувати, але математика працює однаково. Простір ознак — це фундаментальна абстракція ML: усі алгоритми, від лінійної регресії до нейронних мереж, працюють саме з точками у цьому просторі.

Наша мета — знайти функцію $f_\theta(x)$ з параметрами θ, яка для нового, раніше не баченого входу x дасть якомога точніше передбачення $\hat{y} = f_\theta(x)$.

Для програміста це можна уявити як пошук правильних коефіцієнтів у формулі. Наприклад, якщо ви хочете передбачити час проходження рівня (числова величина) — це **регресія**. Якщо потрібно визначити, чи залишиться гравець у грі через місяць (так/ні) — це **класифікація**.

Як зрозуміти, наскільки добре працює модель? Для цього використовують **функцію втрат** (*loss function*) $\mathcal{L}(\theta)$, яка вимірює різницю між передбаченнями моделі та реальними відповідями. Назва «втрати» (*loss*) не випадкова: вона відображає ідею ціни помилки. Кожна неточність моделі — це «втрата» якості, і чим більша різниця між передбаченням і реальністю, тим більша ця втрата. У деяких контекстах використовують також терміни **функція вартості** (*cost function*) — коли акцент на сумарній вартості помилок по всьому набору даних, або **цільова функція** (*objective function*) — коли говорять про оптимізацію загалом. На практиці ці терміни часто використовують як синоніми. Процес навчання — це мінімізація функції втрат:

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta)$$

Тобто ми шукаємо такі параметри θ^*, при яких модель помиляється найменше.

Лінійна регресія

Ідея

Лінійна регресія — це найпростіша модель, яка шукає лінійну залежність між ознаками та цільовою змінною. Якщо ви коли-небудь будували лінію тренду в Excel-таблиці — ви вже використовували лінійну регресію.

Для одної ознаки формула виглядає як рівняння прямої:

$$\hat{y} = w \cdot x + b$$

де w (*weight*) — нахил прямої (вага), b (*bias*) — зсув. Термін **bias** у ML має специфічне значення, відмінне від побутового «упередження». У контексті моделі bias — це параметр зсуву, який дозволяє прямій (або гіперплощині) не проходити через початок координат. Без нього модель була б обмежена: пряма завжди проходила б через точку (0, 0), що рідко відповідає реальним даним. Зверніть увагу: в ML є й інше значення цього слова — **bias моделі** як систематична помилка через надто прості припущення (наприклад, спроба описати параболу прямою лінією). Ці два значення не слід плутати. Для кількох ознак формула узагальнюється до:

$$\hat{y} = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b = \mathbf{w}^T\mathbf{x} + b$$

Ігровий приклад. Ви хочете передбачити шкоду (*damage*) від удару у RPG-грі на основі рівня гравця x_1, сили зброї x_2 та модифікатора навички x_3:

$$\hat{y}_{damage} = w_1 \cdot level + w_2 \cdot weapon_power + w_3 \cdot skill_mod + b$$

Модель сама підбере ваги w_1, w_2, w_3 та зсув b, щоб найкраще відповідати реальним даним бою.

Функція втрат: середньоквадратична помилка (MSE)

Як оцінити якість лінійної регресії? Інтуїтивно — порівняти передбачення з реальністю і покарати модель за різницю. **MSE** (*Mean Squared Error*) — це середнє значення квадратів помилок:

$$\mathcal{L}_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

Чому квадрат? По-перше, він робить усі помилки додатними. По-друге, великі помилки штрафуються непропорційно сильніше — це бажана властивість, бо великий промах зазвичай гірший за кілька маленьких.

Для розробника ігор MSE — як метрика якості балансу: якщо ваша формула шкоди регулярно відхиляється від очікуваної на 50 одиниць — це проблема, і квадрат це підкреслить.

Поліноміальні ознаки та нелінійна регресія

Лінійна регресія обмежена прямими лініями (або площинами у багатовимірному випадку). Але реальні залежності часто нелінійні. Наприклад, шкода від падіння в грі зростає не лінійно з висотою — повільно для малих висот і різко для великих.

Елегантний прийом: замість зміни самого алгоритму можна **розширити простір ознак**, створивши нові ознаки з існуючих. Якщо у нас є ознака x, ми можемо додати x^2, x^3 і навіть комбінації на кшталт $x_1 \cdot x_2$. Такі штучно створені ознаки називаються **поліноміальними ознаками** (*polynomial features*).

Приклад: маючи одну ознаку — висоту падіння h, — ми створюємо розширений вектор ознак:

$$x \to [h,\ h^2,\ h^3]$$

Тепер лінійна регресія на цих нових ознаках стає **поліноміальною регресією**:

$$\hat{y} = w_1 h + w_2 h^2 + w_3 h^3 + b$$

Зверніть увагу: модель залишається лінійною відносно *параметрів* w_1, w_2, w_3 — ми все ще використовуємо зважену суму. Але відносно *вхідних даних* h вона тепер нелінійна — це крива, а не пряма. Ми не змінили алгоритм навчання — лише трансформували дані, перемістивши їх у новий, більш виразний простір ознак.

Цей прийом — **розширення простору ознак** (*feature expansion*) — є надзвичайно потужною ідеєю, яка з'явиться знову у наступній лекції, коли ми будемо говорити про класифікацію та ядрові функції у методі опорних векторів (SVM).

Градієнтний спуск

Інтуїція

Маючи функцію втрат, нам потрібен метод її мінімізації. **Градієнтний спуск** (*gradient descent*) — це ітеративний алгоритм, який крок за кроком зменшує значення функції втрат.

Уявіть, що ви стоїте на горі в густому тумані і хочете спуститися в долину. Ви не бачите всієї гори, але можете відчути нахил під ногами. Стратегія проста: на кожному кроці йдіть в напрямку найбільшого спуску. Саме це і робить градієнтний спуск.

Математично: **градієнт** $\nabla_\theta \mathcal{L}$ — це вектор часткових похідних функції втрат за кожним параметром. Він вказує напрямок найшвидшого зростання функції. Щоб мінімізувати, ми рухаємось у протилежному напрямку:

$$\theta \leftarrow \theta - \alpha \cdot \nabla_\theta \mathcal{L}$$

де α — **швидкість навчання** (*learning rate*) — розмір кроку.

Швидкість навчання

Параметр α критично важливий:

- **Занадто великий** α — модель «перестрибує» мінімум, значення функції втрат починає коливатися або навіть зростати. Це як бігти з гори занадто швидко — промахнетеся повз долину.
- **Занадто малий** α — модель сходиться нестерпно повільно. Тисячі ітерацій можуть не принести суттєвого прогресу.
- **Правильний** α — зазвичай підбирається експериментально. Типові початкові значення: 0.01, 0.001, 0.0001.

Три варіанти градієнтного спуску

Різниця між варіантами — у тому, скільки прикладів використовується для обчислення градієнта на кожному кроці.

Batch Gradient Descent обчислює градієнт на *всьому* наборі даних. Це дає точний напрямок руху, але на великих даних один крок може тривати дуже довго. Якщо у вас мільйон записів ігрової телеметрії — чекати обчислення градієнта по всьому мільйону для кожного кроку непрактично.

Stochastic Gradient Descent (SGD) обчислює градієнт на *одному* випадковому прикладі. Кожен крок дуже швидкий, але напрямок «шумний» — градієнт, обчислений по одному прикладу, може сильно відрізнятися від справжнього. Модель рухається зигзагами, але в середньому — в правильному напрямку. Цей шум іноді навіть корисний: він допомагає вибратися з поганих локальних мінімумів.

Mini-batch Gradient Descent — золота середина. Градієнт обчислюється на невеликому пакеті (*batch*) із B прикладів (типово 32, 64, 128 або 256). Це дає достатньо точний напрямок і при цьому ефективно використовує паралелізм GPU. Саме цей варіант використовується на практиці у переважній більшості сучасних ML-систем.

Варіант	Приклади на крок	Швидкість кроку	Точність градієнта	Використання GPU
Batch	Всі N	Повільно	Висока	Не ефективно

Stochastic	1	Дуже швидко	Низька	Не ефективно
Mini-batch	B (32–256)	Швидко	Достатня	Ефективно

Висновок

У цій лекції ми розглянули фундаментальні компоненти навчання з учителем для задач регресії:

- **Математична постановка** — формалізація задачі навчання з учителем через простір ознак та функцію втрат
- **Лінійна регресія** з MSE — найпростіша модель для передбачення числових величин
- **Поліноміальні ознаки** — потужний трюк розширення простору ознак для моделювання нелінійних залежностей
- **Градієнтний спуск** — універсальний механізм оптимізації, що лежить в основі всього машинного навчання
- **Mini-batch gradient descent** — чому це стандарт індустрії

Ці концепції є фундаментом для розуміння всього подальшого матеріалу. Градієнтний спуск, який ми вивчили для простої лінійної регресії, працює абсолютно так само і для нейронних мереж з мільйонами параметрів — змінюється лише складність обчислення градієнта.

На наступній лекції ми перейдемо до **класифікації** — другої фундаментальної задачі навчання з учителем. Ми вивчимо логістичну регресію, функцію втрат крос-ентропія, як поліноміальні ознаки працюють для нелінійної класифікації, та познайомимося з методом опорних векторів (SVM) і його ядровим трюком.